UTEC
Universidad Tecnológica

Árboles

Introducción a los Árboles

Un árbol es una estructura de datos jerárquica que consiste en nodos conectados por aristas. El nodo superior se llama raíz, y cada nodo puede tener cero o más nodos hijos. Los nodos sin hijos se llaman hojas. Los árboles se utilizan en muchas aplicaciones, como bases de datos, sistemas de archivos y estructuras de búsqueda.

Árboles Binarios

Un árbol binario es un tipo de árbol en el que cada nodo tiene, como máximo, dos hijos, denominados hijo izquierdo y hijo derecho.

Ejemplo de Árbol Binario

```
class Nodo {
    int valor;
    Nodo izquierdo, derecho;

    public Nodo(int item) {
        valor = item;
        izquierdo = derecho = null;
    }
}

class ArbolBinario {
    Nodo raiz;

    // Constructor
    ArbolBinario() {
        raiz = null;
    }

    // Método para recorrer el árbol en orden (in-order traversal)
    void enOrden(Nodo nodo) {
        if (nodo != null) {
            enOrden(nodo.izquierdo);
```

UTEC
Universidad Tecnológica

```
            System.out.print(nodo.valor + " ");
            enOrden(nodo.derecho);
        }
    }

    // Método principal
    public static void main(String[] args) {
        ArbolBinario arbol = new ArbolBinario();
        arbol.raiz = new Nodo(1);
        arbol.raiz.izquierdo = new Nodo(2);
        arbol.raiz.derecho = new Nodo(3);
        arbol.raiz.izquierdo.izquierdo = new Nodo(4);
        arbol.raiz.izquierdo.derecho = new Nodo(5);

        System.out.println("Recorrido en orden del árbol binario:");
        arbol.enOrden(arbol.raiz);
    }
}
```

Árboles de Búsqueda Binaria (BSTs)

Un árbol de búsqueda binaria es un tipo especial de árbol binario que mantiene el orden de los elementos. Para cada nodo, todos los nodos en el subárbol izquierdo tienen valores menores, y todos los nodos en el subárbol derecho tienen valores mayores.

Ejemplo de BST

```
class NodoBST {
    int valor;
    NodoBST izquierdo, derecho;

    public NodoBST(int item) {
        valor = item;
        izquierdo = derecho = null;
    }
}

class ArbolBST {
```

UTEC
Universidad Tecnológica

```
    NodoBST raiz;

    // Constructor
    ArbolBST() {
        raiz = null;
    }

    // Método para insertar un nuevo nodo en el BST
    void insertar(int valor) {
        raiz = insertarRec(raiz, valor);
    }

    // Función recursiva para insertar un nuevo valor en el BST
    NodoBST insertarRec(NodoBST raiz, int valor) {
        if (raiz == null) {
            raiz = new NodoBST(valor);
            return raiz;
        }

        if (valor < raiz.valor)
            raiz.izquierdo = insertarRec(raiz.izquierdo, valor);
        else if (valor > raiz.valor)
            raiz.derecho = insertarRec(raiz.derecho, valor);

        return raiz;
    }

    // Método para realizar una búsqueda en el BST
    boolean buscar(int valor) {
        return buscarRec(raiz, valor);
    }

    // Función recursiva para buscar un valor en el BST
    boolean buscarRec(NodoBST raiz, int valor) {
        if (raiz == null) {
            return false;
        }

        if (raiz.valor == valor) {
            return true;
        }
```

UTEC
Universidad Tecnológica

```
        if (valor < raiz.valor) {
            return buscarRec(raiz.izquierdo, valor);
        } else {
            return buscarRec(raiz.derecho, valor);
        }
    }

    // Método principal
    public static void main(String[] args) {
        ArbolBST arbol = new ArbolBST();

        arbol.insertar(50);
        arbol.insertar(30);
        arbol.insertar(20);
        arbol.insertar(40);
        arbol.insertar(70);
        arbol.insertar(60);
        arbol.insertar(80);

        System.out.println("El valor 40 está en el árbol: " +
arbol.buscar(40));
        System.out.println("El valor 90 está en el árbol: " +
arbol.buscar(90));
    }
}
```

Operaciones Básicas: Inserción, Eliminación, Búsqueda

Las operaciones básicas en un árbol de búsqueda binaria incluyen la inserción de un nuevo nodo, la eliminación de un nodo existente y la búsqueda de un valor.

Inserción

La inserción implica agregar un nuevo nodo en la posición correcta de acuerdo con las reglas del BST.

Eliminación

La eliminación de un nodo puede ser compleja y depende de si el nodo a eliminar tiene cero, uno o dos hijos.

UTEC
Universidad Tecnológica

```
NodoBST eliminar(NodoBST raiz, int valor) {
    if (raiz == null) {
        return raiz;
    }

    if (valor < raiz.valor) {
        raiz.izquierdo = eliminar(raiz.izquierdo, valor);
    } else if (valor > raiz.valor) {
        raiz.derecho = eliminar(raiz.derecho, valor);
    } else {
        // Nodo con un solo hijo o sin hijos
        if (raiz.izquierdo == null) {
            return raiz.derecho;
        } else if (raiz.derecho == null) {
            return raiz.izquierdo;
        }

        // Nodo con dos hijos: obtener el sucesor (menor en el subárbol
derecho)
        raiz.valor = valorMinimo(raiz.derecho);
        raiz.derecho = eliminar(raiz.derecho, raiz.valor);
    }

    return raiz;
}

int valorMinimo(NodoBST raiz) {
    int minValor = raiz.valor;
    while (raiz.izquierdo != null) {
        minValor = raiz.izquierdo.valor;
        raiz = raiz.izquierdo;
    }
    return minValor;
}
```

Búsqueda

La búsqueda en un BST sigue las reglas de comparación para encontrar un valor.

UTEC
Universidad Tecnológica

```java
boolean buscarRec(NodoBST raiz, int valor) {
    if (raiz == null) {
        return false;
    }

    if (raiz.valor == valor) {
        return true;
    }

    if (valor < raiz.valor) {
        return buscarRec(raiz.izquierdo, valor);
    } else {
        return buscarRec(raiz.derecho, valor);
    }
}
```

Conceptos de Árboles Balanceados

Un árbol balanceado es un árbol en el que la altura de los subárboles izquierdo y derecho de cualquier nodo difiere en no más de una unidad. Los árboles balanceados garantizan que las operaciones de inserción, eliminación y búsqueda se realicen en tiempo logarítmico en el peor de los casos.

Ejemplo: Árbol AVL

Los árboles AVL (Adelson-Velsky y Landis) son un tipo de árbol binario de búsqueda balanceado en el que la diferencia de alturas entre los subárboles izquierdo y derecho de cualquier nodo es, como máximo, uno.

```java
class NodoAVL {
    int valor, altura;
    NodoAVL izquierdo, derecho;

    NodoAVL(int d) {
        valor = d;
        altura = 1;
    }
}
```

UTEC
Universidad Tecnológica

```
class ArbolAVL {
    NodoAVL raiz;

    // Obtener la altura de un nodo
    int altura(NodoAVL N) {
        if (N == null)
            return 0;
        return N.altura;
    }

    // Rotación simple a la derecha
    NodoAVL rotacionDerecha(NodoAVL y) {
        NodoAVL x = y.izquierdo;
        NodoAVL T2 = x.derecho;

        x.derecho = y;
        y.izquierdo = T2;

        y.altura = Math.max(altura(y.izquierdo), altura(y.derecho)) + 1;
        x.altura = Math.max(altura(x.izquierdo), altura(x.derecho)) + 1;

        return x;
    }

    // Rotación simple a la izquierda
    NodoAVL rotacionIzquierda(NodoAVL x) {
        NodoAVL y = x.derecho;
        NodoAVL T2 = y.izquierdo;

        y.izquierdo = x;
        x.derecho = T2;

        x.altura = Math.max(altura(x.izquierdo), altura(x.derecho)) + 1;
        y.altura = Math.max(altura(y.izquierdo), altura(y.derecho)) + 1;

        return y;
    }

    // Obtener el factor de balance de un nodo
    int obtenerBalance(NodoAVL N) {
        if (N == null)
```

UTEC
Universidad Tecnológica

```
            return 0;
        return altura(N.izquierdo) - altura(N.derecho);
    }

    // Insertar un nodo en el árbol AVL
    NodoAVL insertar(NodoAVL nodo, int valor) {
        if (nodo == null)
            return new NodoAVL(valor);

        if (valor < nodo.valor)
            nodo.izquierdo = insertar(nodo.izquierdo, valor);
        else if (valor > nodo.valor)
            nodo.derecho = insertar(nodo.derecho, valor);
        else // Valores duplicados no permitidos
            return nodo;

        nodo.altura = 1 + Math.max(altura(nodo.izquierdo),
altura(nodo.derecho));

        int balance = obtenerBalance(nodo);

        if (balance > 1 && valor < nodo.izquierdo.valor)
            return rotacionDerecha(nodo);

        if (balance < -1 && valor > nodo.derecho.valor)
            return rotacionIzquierda(nodo);

        if (balance > 1 && valor > nodo.izquierdo.valor) {
            nodo.izquierdo = rotacionIzquierda(nodo.izquierdo);
            return rotacionDerecha(nodo);
        }

        if (balance < -1 && valor < nodo.derecho.valor) {
            nodo.derecho = rotacionDerecha(nodo.derecho);
            return rotacionIzquierda(nodo);
        }

        return nodo;
    }

    // Método principal
    public static void main(String[] args) {
```

UTEC
Universidad Tecnológica

```
        ArbolAVL arbol = new ArbolAVL();

        arbol.raiz = arbol.insertar(arbol.raiz, 10);
        arbol.raiz = arbol.insertar(arbol.raiz, 20);
        arbol.raiz = arbol.insertar(arbol.raiz, 30);
        arbol.raiz = arbol.insertar(arbol.raiz, 40);
        arbol.raiz = arbol.insertar(arbol.raiz, 50);
        arbol.raiz = arbol.insertar(arbol.raiz, 25);

        System.out.println("Arbol AVL construido.");
    }
}
```

En Java, la API de colecciones proporciona varias implementaciones de estructuras de datos basadas en árboles. Las más comunes son TreeSet y TreeMap, que forman parte del paquete java.util.

TreeSet

TreeSet es una implementación de la interfaz Set que almacena sus elementos en un árbol de búsqueda binaria balanceado. Esto garantiza que las operaciones básicas como búsqueda, inserción y eliminación tengan una complejidad de tiempo logarítmica \(O(log n)\).

Ejemplo de uso de TreeSet

```
import java.util.TreeSet;

public class EjemploTreeSet {
    public static void main(String[] args) {
        // Crear un TreeSet
        TreeSet<Integer> numeros = new TreeSet<>();

        // Agregar elementos
        numeros.add(5);
        numeros.add(1);
        numeros.add(10);
        numeros.add(7);
```

UTEC
Universidad Tecnológica

```
        // Mostrar elementos (ordenados)
        System.out.println("Elementos en TreeSet: " + numeros);

        // Buscar elementos
        System.out.println("Contiene 5? " + numeros.contains(5));

        // Eliminar un elemento
        numeros.remove(10);
        System.out.println("Elementos después de eliminar 10: " +
numeros);

        // Obtener el primer y último elemento
        System.out.println("Primer elemento: " + numeros.first());
        System.out.println("Último elemento: " + numeros.last());
    }
}
```

TreeMap

TreeMap es una implementación de la interfaz Map que almacena sus pares clave-valor en un árbol de búsqueda binaria balanceado. Esto asegura que las operaciones de búsqueda, inserción y eliminación tengan una complejidad de tiempo logarítmica \(O(log n)\).

Ejemplo de uso de TreeMap

```
import java.util.TreeMap;

public class EjemploTreeMap {
    public static void main(String[] args) {
        // Crear un TreeMap
        TreeMap<Integer, String> estudiantes = new TreeMap<>();

        // Agregar elementos
        estudiantes.put(1, "Juan");
        estudiantes.put(3, "Ana");
        estudiantes.put(2, "Luis");

        // Mostrar elementos (ordenados por clave)
```

UTEC
Universidad Tecnológica

```
        System.out.println("Elementos en TreeMap: " + estudiantes);

        // Buscar elementos
        System.out.println("Estudiante con ID 2: " +
estudiantes.get(2));

        // Eliminar un elemento
        estudiantes.remove(3);
        System.out.println("Elementos después de eliminar ID 3: " +
estudiantes);

        // Obtener el primer y último elemento
        System.out.println("Primera entrada: " +
estudiantes.firstEntry());
        System.out.println("Última entrada: " +
estudiantes.lastEntry());
    }
}
```

NavigableSet y NavigableMap

TreeSet y TreeMap también implementan las interfaces NavigableSet y NavigableMap, respectivamente. Estas interfaces proporcionan métodos adicionales para navegación y búsqueda, como lower(), higher(), floor(), y ceiling().

Ejemplo de uso de NavigableSet y NavigableMap

```
import java.util.NavigableSet;
import java.util.NavigableMap;
import java.util.TreeSet;
import java.util.TreeMap;

public class EjemploNavigable {
    public static void main(String[] args) {
        // Ejemplo de NavigableSet con TreeSet
        NavigableSet<Integer> numeros = new TreeSet<>();
        numeros.add(1);
        numeros.add(3);
```

UTEC
Universidad Tecnológica

```
        numeros.add(5);
        numeros.add(7);

        System.out.println("Mayor que 3: " + numeros.higher(3)); // 5
        System.out.println("Menor que 3: " + numeros.lower(3)); // 1
        System.out.println("Mayor o igual a 3: " + numeros.ceiling(3));
// 3
        System.out.println("Menor o igual a 3: " + numeros.floor(3)); //
3

        // Ejemplo de NavigableMap con TreeMap
        NavigableMap<Integer, String> estudiantes = new TreeMap<>();
        estudiantes.put(1, "Juan");
        estudiantes.put(3, "Ana");
        estudiantes.put(5, "Luis");

        System.out.println("Entrada mayor que 3: " +
estudiantes.higherEntry(3)); // 5=Luis
        System.out.println("Entrada menor que 3: " +
estudiantes.lowerEntry(3)); // 1=Juan
        System.out.println("Entrada mayor o igual a 3: " +
estudiantes.ceilingEntry(3)); // 3=Ana
        System.out.println("Entrada menor o igual a 3: " +
estudiantes.floorEntry(3)); // 3=Ana
    }
}
```

Resumen

- TreeSet: Implementa un conjunto ordenado utilizando un árbol de búsqueda binaria balanceado.
- TreeMap: Implementa un mapa ordenado utilizando un árbol de búsqueda binaria balanceado.
- NavigableSet y NavigableMap: Proporcionan métodos adicionales para navegación y búsqueda dentro de conjuntos y mapas ordenados.